3 Looping

A **loop** is a programming instruction that repeats until a specific condition is reached. One **iteration** is when the code block within a loop is executed one time. Depending on your specific programming situation, you may need different loop-structures that execute the code blocks within the loop on the basis of different conditions.

By the end of this section you should be able to use the three common types of loops:

- `for` loops
- `while` loops
- `repeat` loops

3.1 `for` Loops

A `for` loop is perhaps the most instinctive way to iteratively perform an action time and again. However, as we shall continue to see throughout this course, they are not generally necessary and it may be better to use the `map()` function instead which we will learn to use later on in the course. They do of course still have their place, so it's definitely worth knowing how to use them.

In particular, they are useful when the number of iterations are known before entering the loop.

3.1.1 Example

Let's suppose we observed a bunch of p-values from different endpoints in a study and we want to know whether each one is less than 0.05 or not.

This is not the best way of doing it, this example is just for illustration purposes.

First, the p-values:

```
# Read in some p-values
p_values <- c( 0.0223, 0.1508, 0.0108, 0.0372, 0.1164, 0.1373, 0.1671, 0.0155, 0.1852, 0.1293)
```

Now let's use a `for` loop to check whether each p-value is < 0.05. For now we just print this to screen using `cat()`.

To call a `for` loop, we use the `for` control. We provide the control with an iterator and a vector to iterate over. Curly brackets, `{}` control where the loop starts and ends. The vector that we loop through can be of any mode/type. R simply moves along it such that the number of loops corresponds to the length of the vector. Upon reaching the end of the first loop, R automatically moves along to the next element of the vector and begins another loop.

```
for(i in p_values){
  trial_outcome <- if_else(i < 0.05, "success", "failure")
  cat("p-value: ", i, ", Outcome: ", trial_outcome, "\n", sep = "")
}
```

```
p-value: 0.0223, Outcome: success
p-value: 0.1508, Outcome: failure
p-value: 0.0108, Outcome: success
p-value: 0.0372, Outcome: success
p-value: 0.1164, Outcome: failure
p-value: 0.1373, Outcome: failure
p-value: 0.1671, Outcome: failure
p-value: 0.0155, Outcome: success
p-value: 0.1852, Outcome: failure
p-value: 0.1293, Outcome: failure
```

3.1.2 Vectorisation

The use of the `for` loop in this example was for illustration purposes only. Since R is vectorised, the following code is faster to type and *much, much faster* to run. It also creates a vector that we can use.

```
# Loop through the p-values determining if the trial was a success.
trial_outcome <- p_values < 0.05
trial_outcome
```

```
 [1]  TRUE FALSE  TRUE  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE
```

```
# How many were a success?
sum(trial_outcome)
```

```
[1] 4
```

`for` loops can be used to generate vectors but there's nearly always a better way so this functionality is not shown!

3.1.3 The `seq_along()` and `seq_len()` Functions

The `seq_along()` and `seq_len()` functions are useful when looping over vectors of unknown length. They avoid an infamous issue caused by zero-length vectors.

```
# Let's define a rare group of people, i.e. a group with no people in it
rare_group <- vector(length = 0)
for(i in 1:length(rare_group)){
  cat(i, "\n")
}
```

```
1
0
```

We may have expected nothing to be returned here, since our group was empty. However, we actually looped over the vector `1:0` which has two elements, `1` and `0`. This resulted in two (unwanted) loops.

To avoid this, we can use the `seq_along()` function. This function is equivalent to `1:length(x)` if `length(x) > 0` but returns an empty vector if `length(x) == 0`.

```
for(i in seq_along(rare_group)){
  cat(i, "\n")
}
```

The `seq_len` function is similar but instead of passing the vector we pass its length.

```
# Our example of no length
for(i in seq_len(0)){
  cat(i, "\n")
}
# A more useful application
n_sims <- c(2,1,3,4,8)
for(i in seq_len(n_sims)){
  cat(i, "\n")
}
```

```
Warning in seq_len(n_sims): first element used of 'length.out' argument
```

```
1
2
```

3.2 `while` Loops

The `while` loops repeat a code block as long as a certain logical condition is TRUE. They are typically used when we don't know the exact number of times we need our R code to be executed.

To call a `while` loop, we use the `while` control. We start with an initialization block followed by the control with a logical condition. Curly brackets, `{}` control where the loop starts and ends.

The implementation of a `while` loop is actually very similar to the implementation of an `if` statement. The difference is that, in a `while` loop, the single condition that we provide is continuously evaluated at the end of each loop. Below is an simple example of a `while` loop.

```
# Initialize i
i <- 1

while (i < 6) {
print(i)
i = i+1
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

Always remember to initialize the variable that makes your condition and make sure it is incrementing.

In the above example, `i` is initialized to 1. The single condition we are testing is `i < 6` which evaluates to TRUE since 1 is less than 6. So, the body of the loop is entered and `i` is printed and incremented.

Incrementing `i` is important so that the exit condition is eventually met, and the loop stops. Failing to do so will result in an infinite loop. In this example, the loop will continue to iterate and add +1 to `i` each time. This will continue until `i = 6`. At this point, the condition `i < 6` will evaluate to FALSE and the while loop will exit.

We should always try to avoid using `while` loops if at all possible, as they are particularly slow in R.

3.3 `repeat` Loops

The `repeat` loops repeat a code block until a break condition is fulfilled. This break condition marks the end of the loop.

These loops follow a similar logic to `while` loops in the sense that they can be used when the user does not know the exact number of times the R code should be repeated. However, `repeat` loops are not as popular as while loops.

```r
#Initialisation
x_repeat <- 0

# Head of repeat-loop
repeat{
  # Body of repeat-loop
  x_repeat <- x_repeat + 1
  print(x_repeat)

  # Break condition of repeat-loop
  if(x_repeat >= 10) {

    break
  }
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6
```

```
[1] 7
[1] 8
[1] 9
[1] 10
```

3.4 EXERCISE

1. In the `vs.sas7bdat` data, select the first 5 subjects, determine whose height is greater than 170 cm and weight is greater than 90 kg. (Try out both the `for` loop and the vectorisation technique.)
2. In the `dm.sas7bdat` data, create a new column `Agegp` which categorizes the age into three groups: ≤18 years, 19 - 64 years, ≥ 65 years (Hint: Don't forget you have learned the `case_when()` function!)

The end result should look something like the below:

Show 5 entries Search:

	USUBJID	AGE	SEX	COUNTRY	ARM	Agegp
1	STD123456:000001	32	F	UK	Comparator	19 - 64 years
2	STD123456:000002	28	M	FRA	Comparator	19 - 64 years
3	STD123456:000003	55	M	USA	Comparator	19 - 64 years
4	STD123456:000004	35	F	GER	Comparator	19 - 64 years
5	STD123456:000005	30	F	IRE	Comparator	19 - 64 years

Showing 1 to 5 of 30 entries Previous 1 2 3 4 5 6 Next